

Last time: RIs and Private Attributes

- ❑ **Public attributes:** easy to access, easy to change, hard to change implementation afterwards
- ❑ **Private attributes:** lets us change implementation details
- ❑ **Representation invariants:** a condition that must hold true for all instances of a class

Inheritance

CSC148 INTRODUCTION TO COMPUTER SCIENCE

Related Readings: [3.5](#)–3.9

This Lecture: inheritance & interfaces!

By the end of this week's lectures, you should be able to:



Inheritance

- ☐ Define, identify & produce examples of **inheritance**, including the 4 types of method inheritance
- ☐ Translate Python code that uses inheritance into a **memory model**
- ☐ Explain whether inheritance is appropriate for a given problem
- ☐ Design a class hierarchy for a given problem, and justify the design decisions



Interfaces

- ☐ Define **abstract class** & **polymorphism**
- ☐ Identify & create code that uses these concepts



Special methods

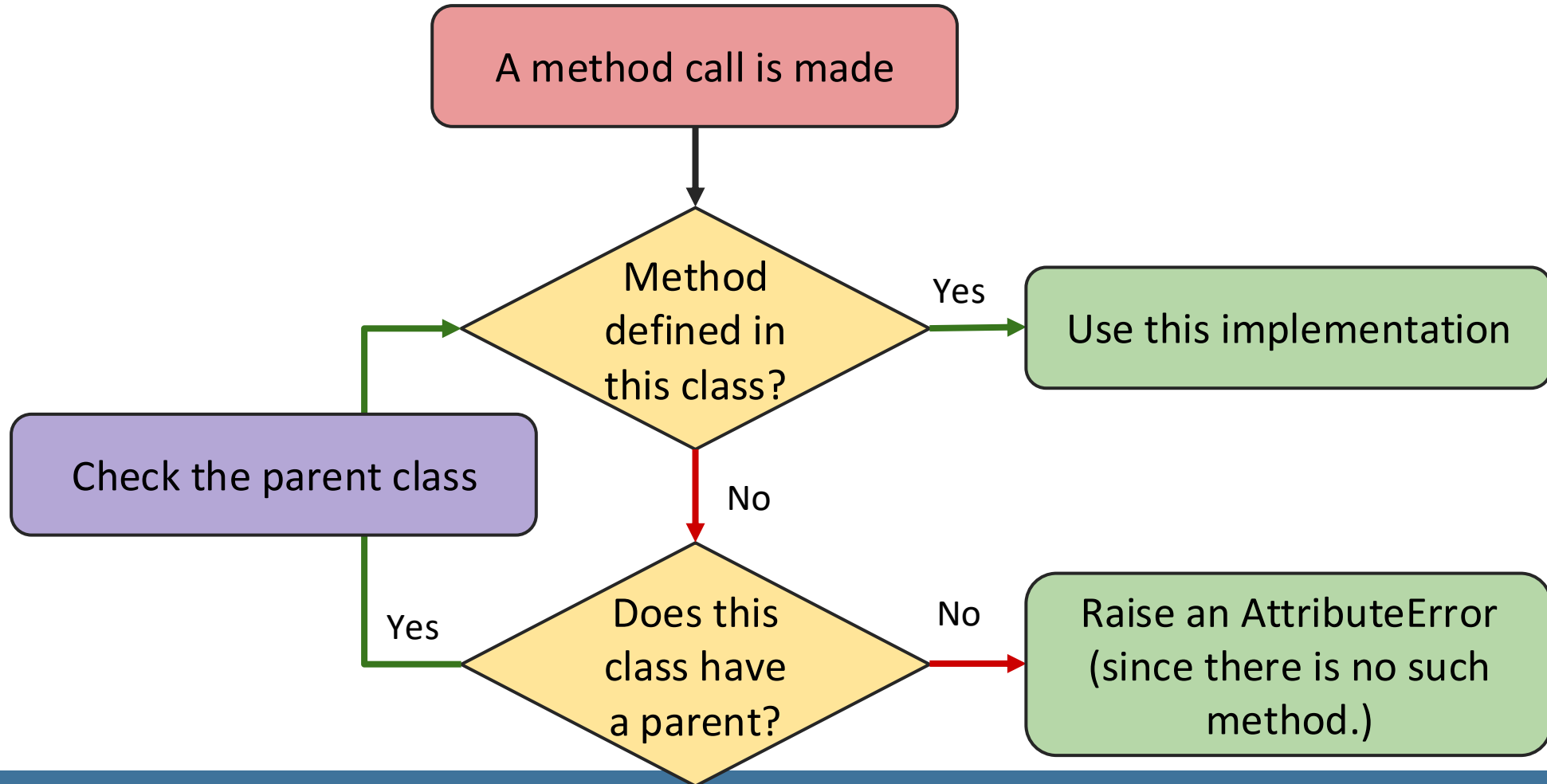
- ☐ Define and implement **special methods**
- ☐ Explain their connection to the **object** class

Recap: Inheritance

- ❑ **Child class:** subclass, derived class
 - ❑ Inherits from a parent
 - ❑ Has access to everything defined in the parent
- ❑ **Parent class:** superclass, base class
 - ❑ The class that is inherited from
- ❑ **Abstract class:** a class that is not to be instantiated
 - ❑ Some methods **raise NotImplementedError**

```
class ChildName (ParentName) :
```

Recap: Inheritance



Draw a memory model diagram

```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x

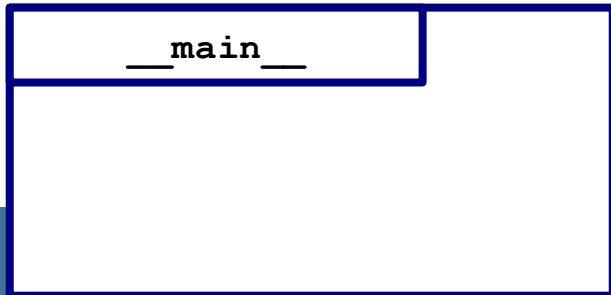
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10

if __name__ == '__main__':
    c = Child(5)
```

```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

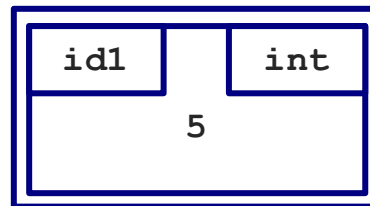
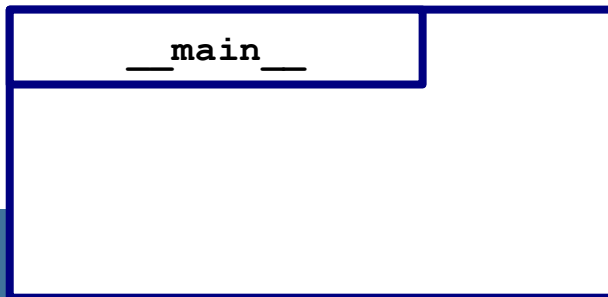
```
if __name__ == '__main__':
    ➡ c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

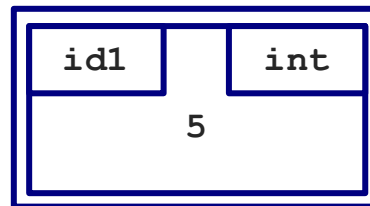
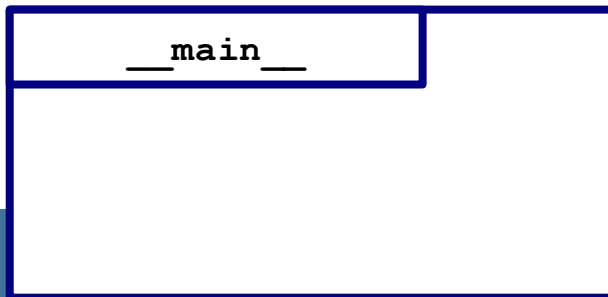
```
if __name__ == '__main__':
    c = Child(5)
```




```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

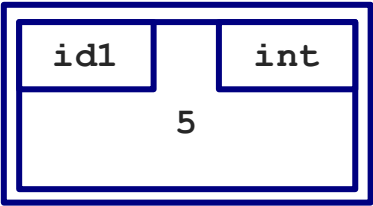
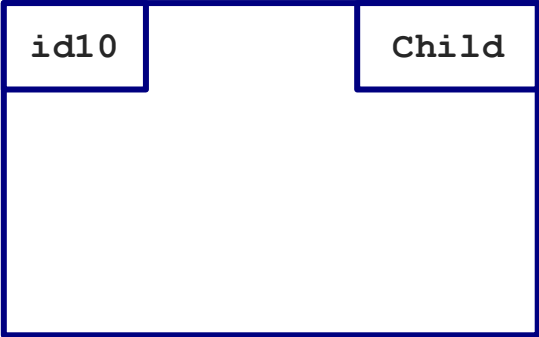
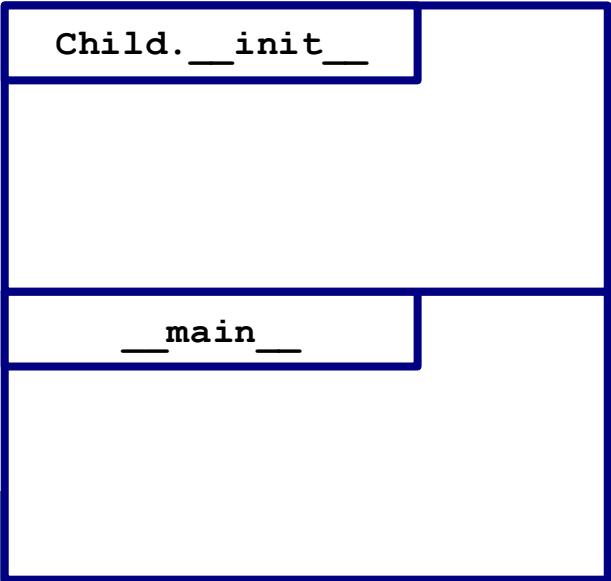
```
if __name__ == '__main__':
    c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

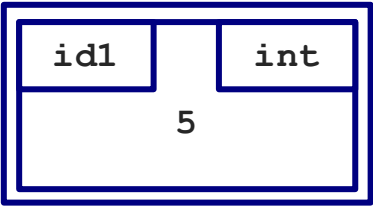
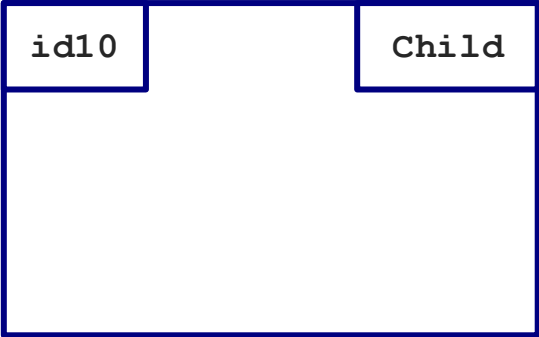
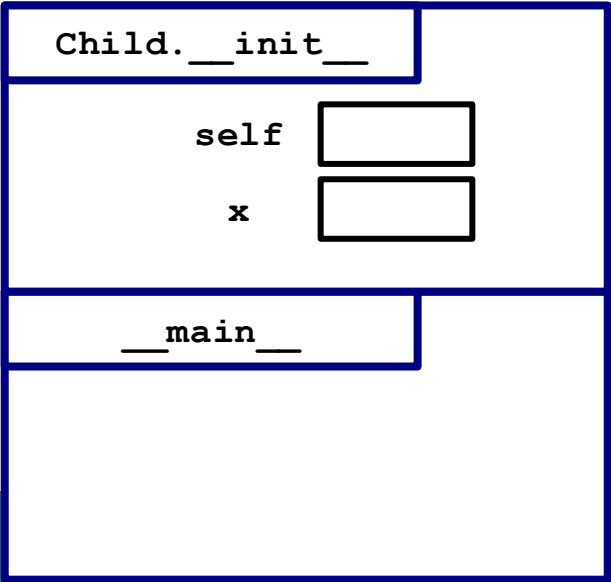
```
if __name__ == '__main__':
    c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

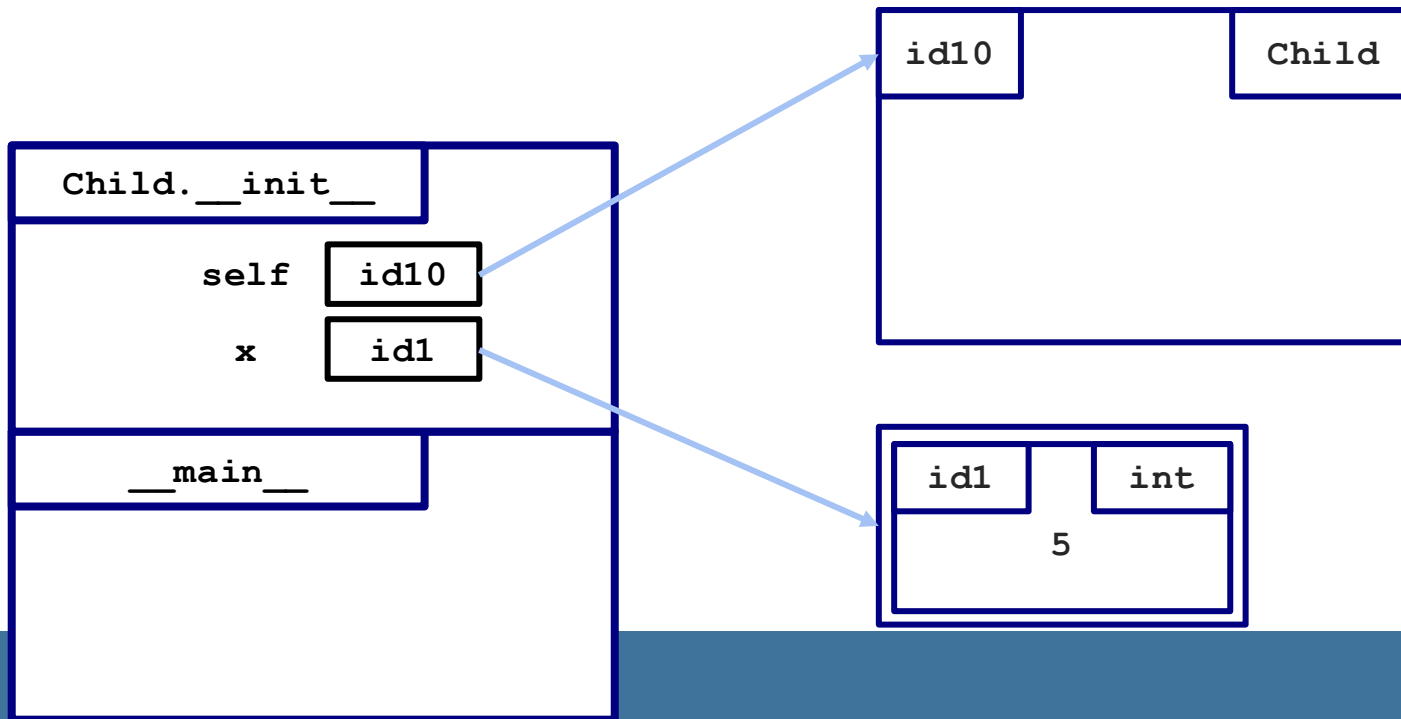
```
if __name__ == '__main__':
    c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    → def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

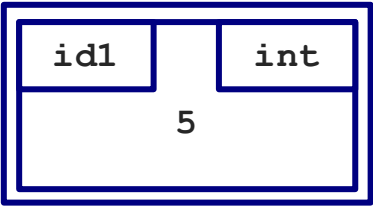
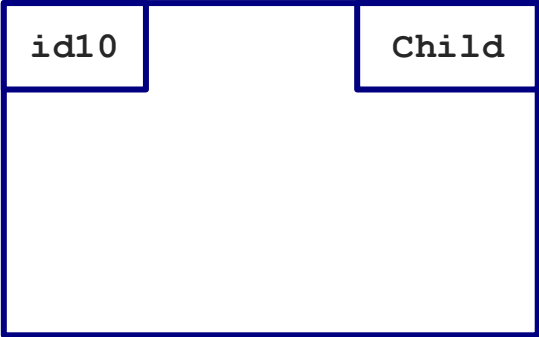
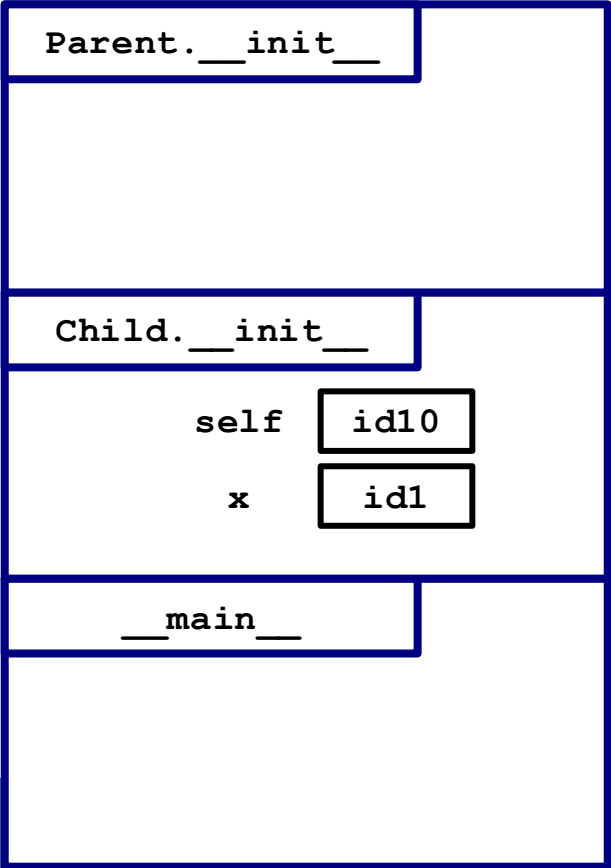
```
if __name__ == '__main__':
    → c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        → Parent.__init__(self, x)
        self.y = 10
```

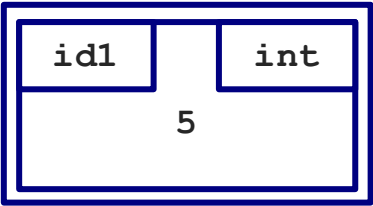
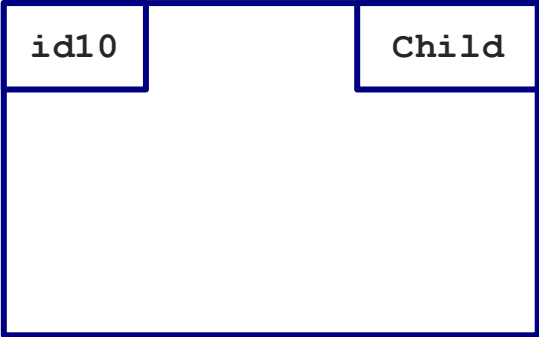
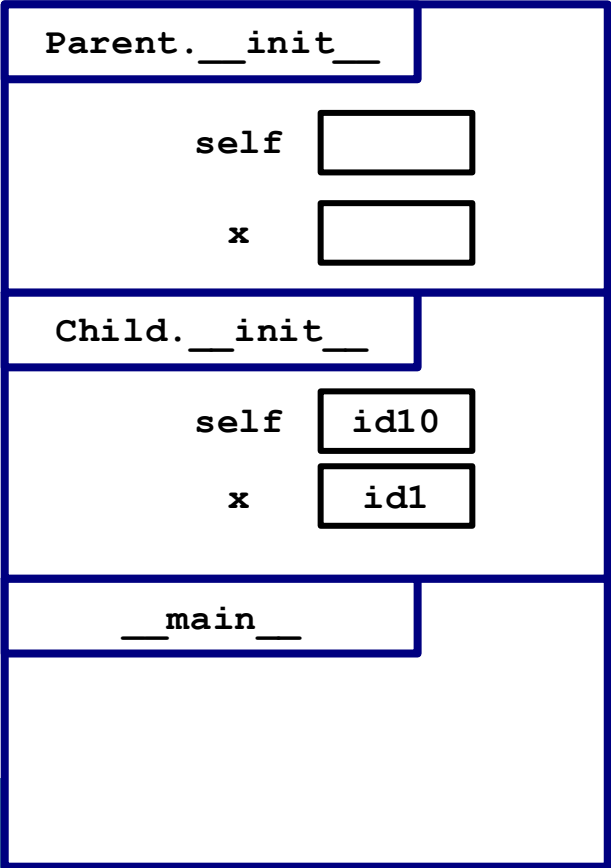
```
if __name__ == '__main__':
    → c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

```
if __name__ == '__main__':
    c = Child(5)
```

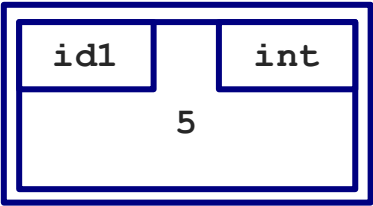
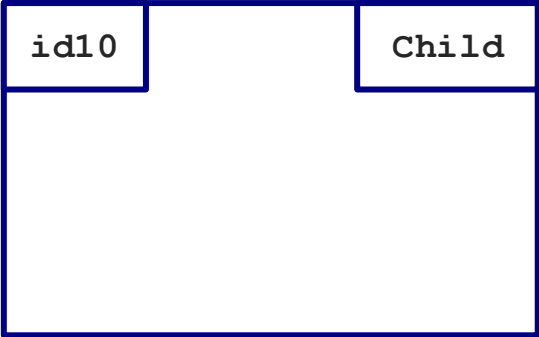
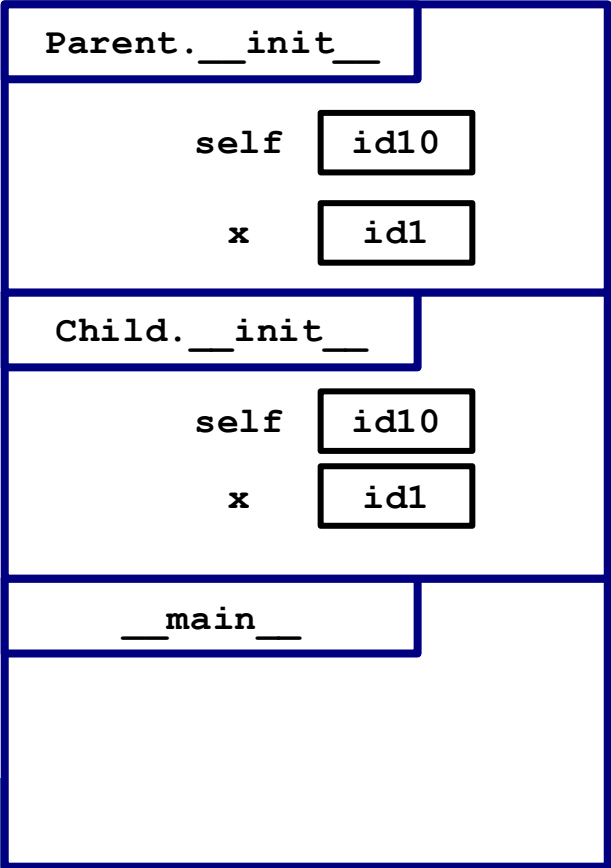


```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

```
if __name__ == '__main__':
    c = Child(5)
```

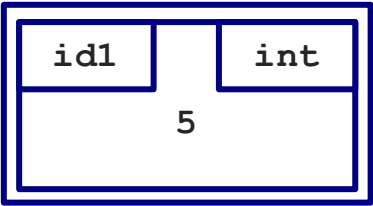
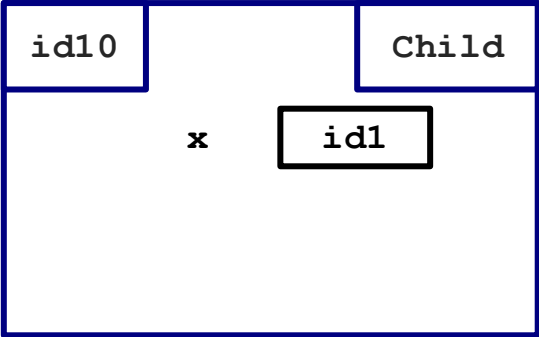
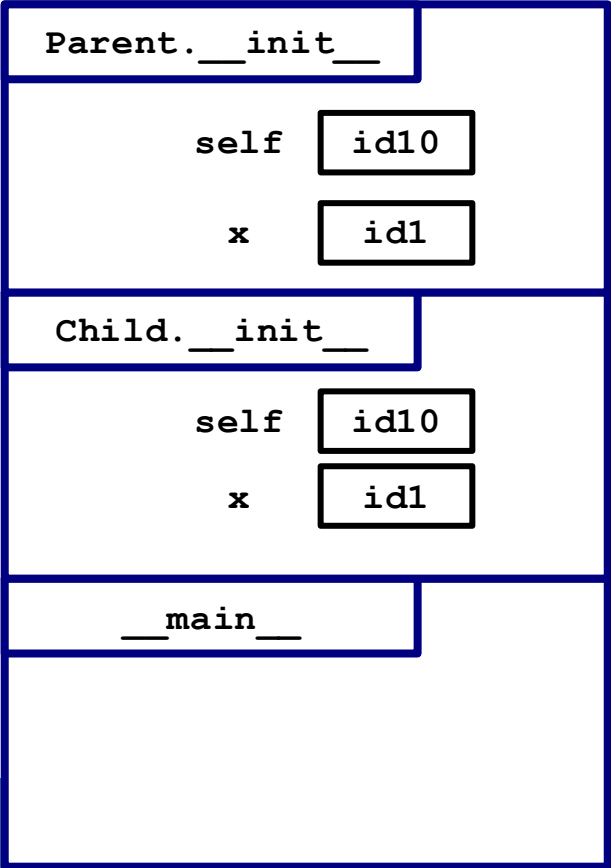
Think of this as just a normal function call: **self** is just an argument that we pass in!



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

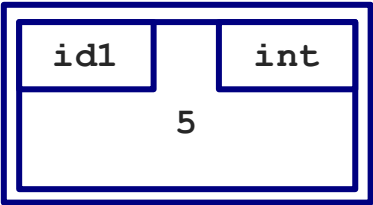
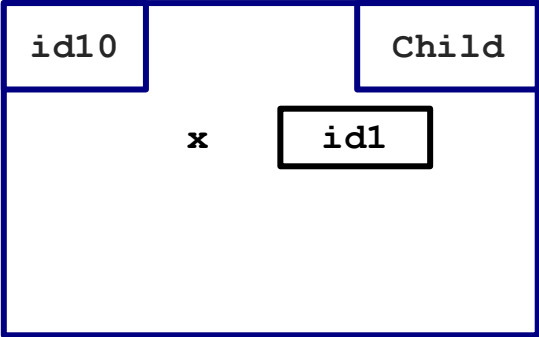
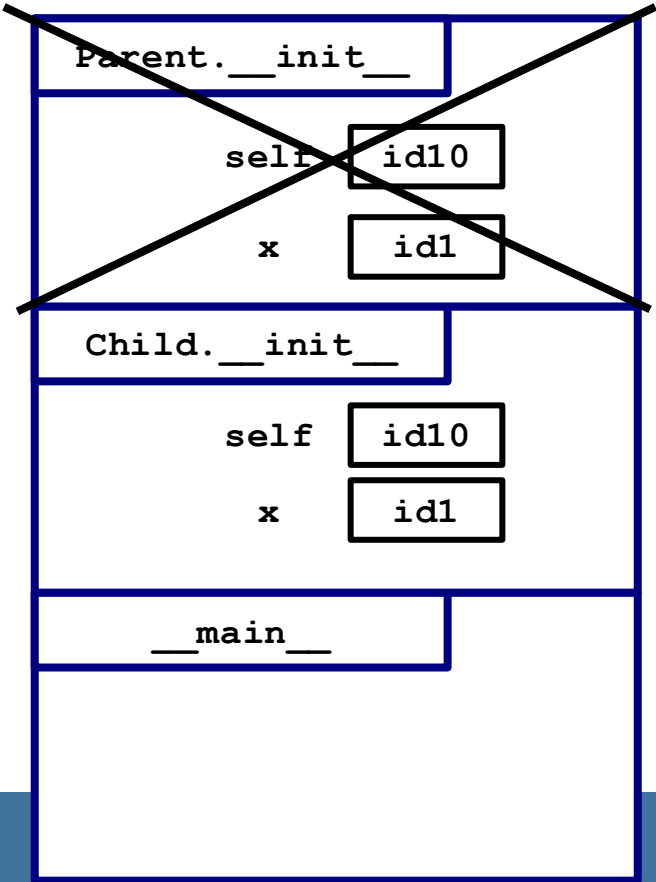
```
if __name__ == '__main__':
    c = Child(5)
```




```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        → Parent.__init__(self, x)
        self.y = 10
```

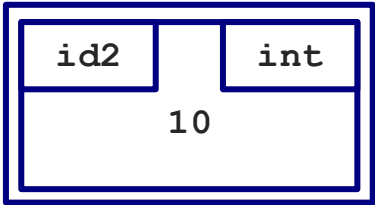
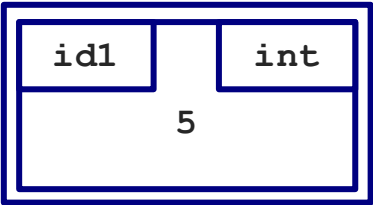
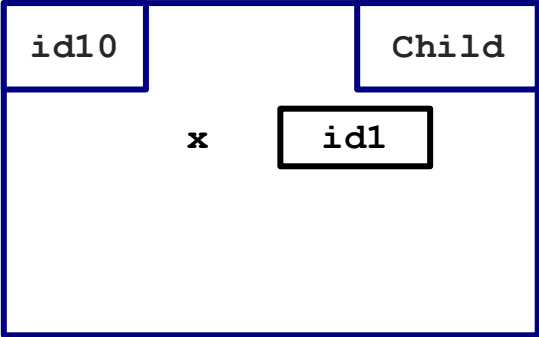
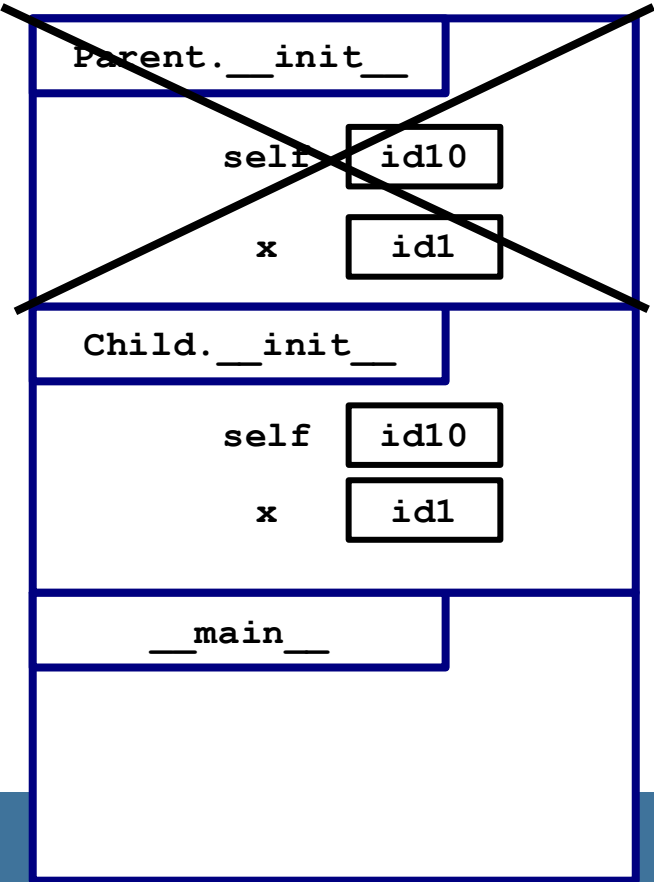
```
if __name__ == '__main__':
    → c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

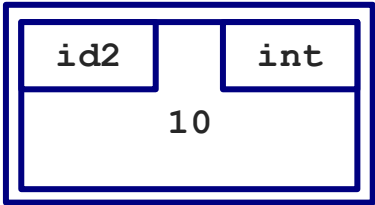
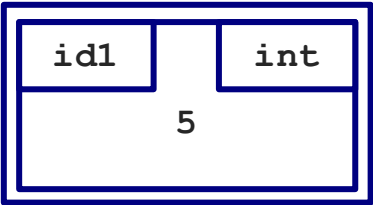
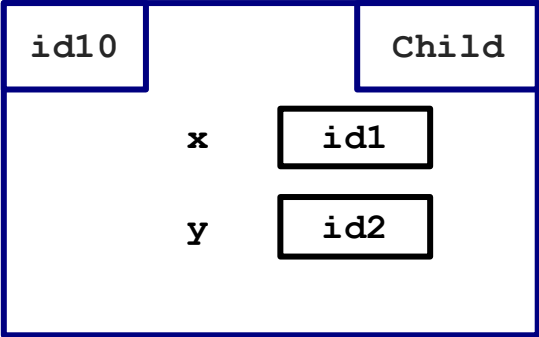
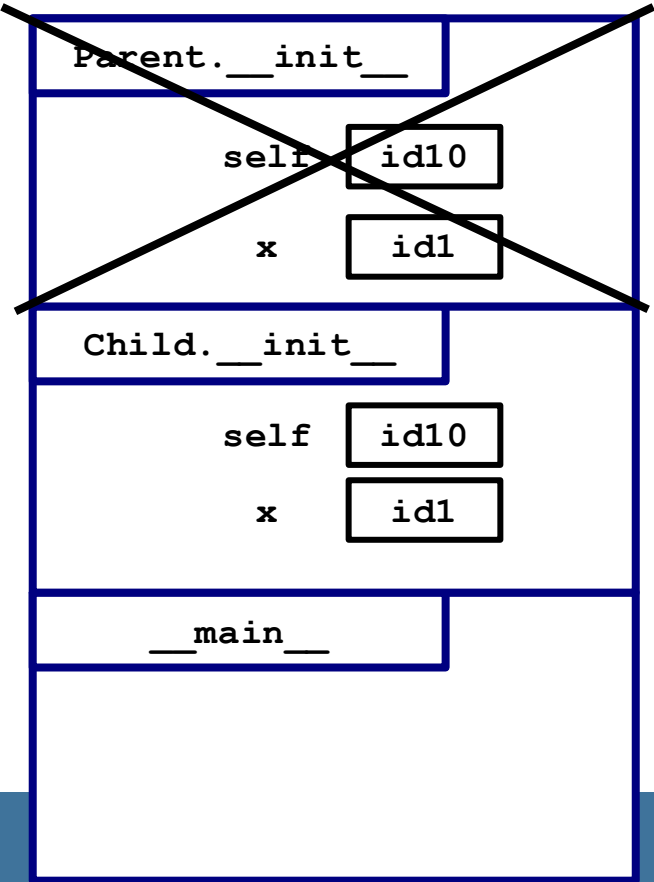
```
if __name__ == '__main__':
    c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

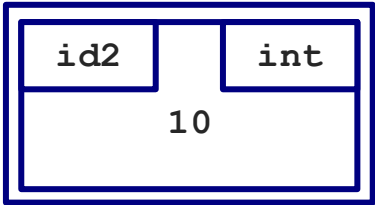
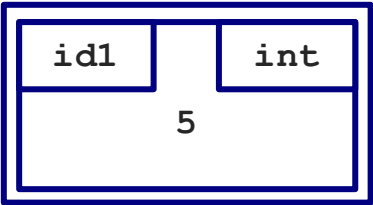
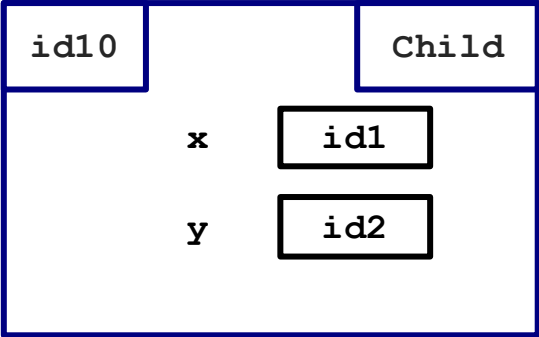
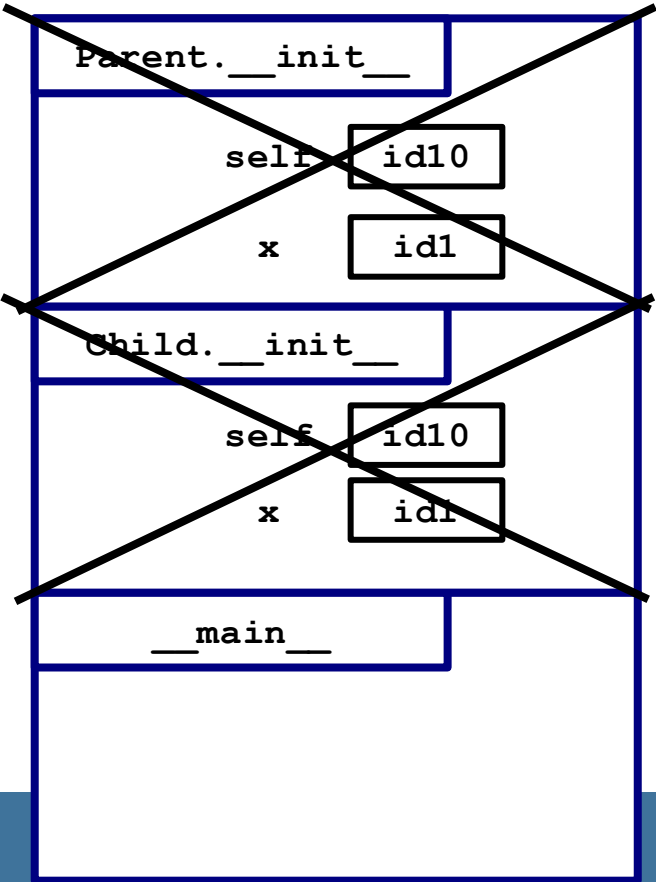
```
if __name__ == '__main__':
    c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

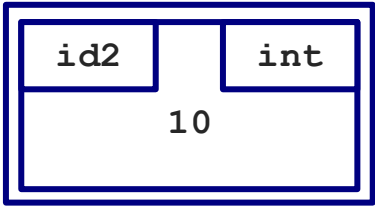
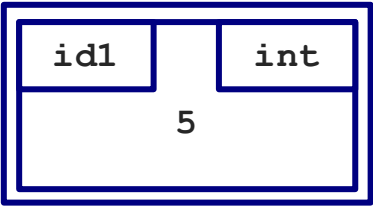
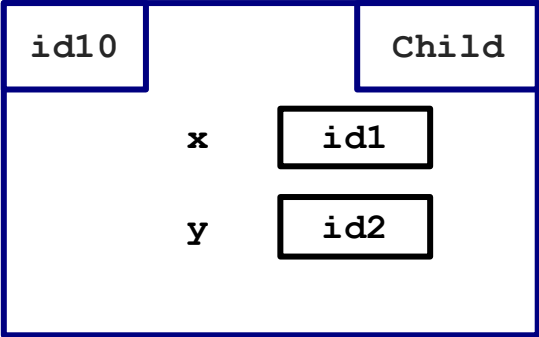
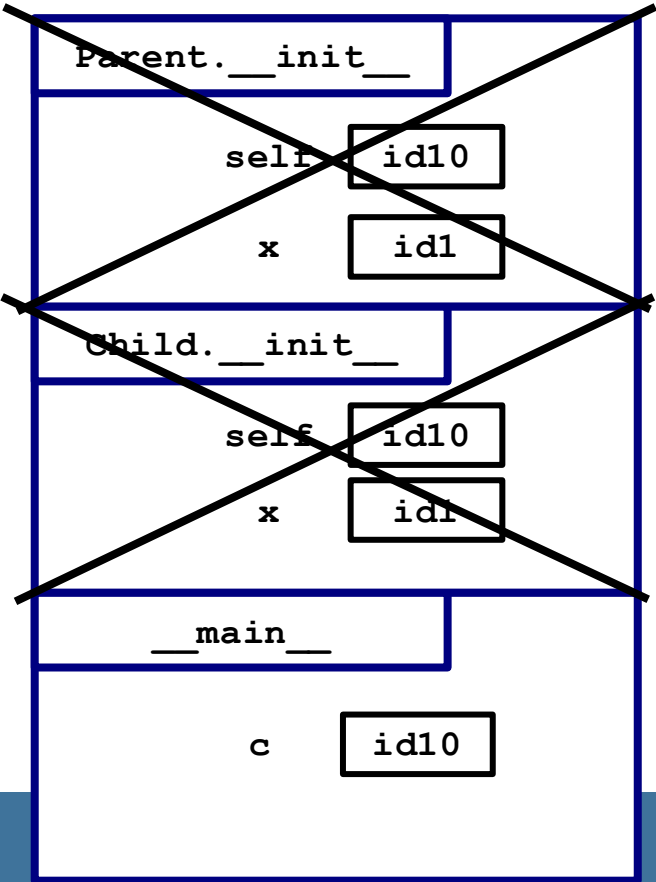
```
if __name__ == '__main__':
    ➡ c = Child(5)
```



```
class Parent:
    def __init__(self, x: int) -> None:
        self.x = x
```

```
class Child(Parent):
    def __init__(self, x: int) -> None:
        Parent.__init__(self, x)
        self.y = 10
```

```
if __name__ == '__main__':
    ➔ c = Child(5)
```

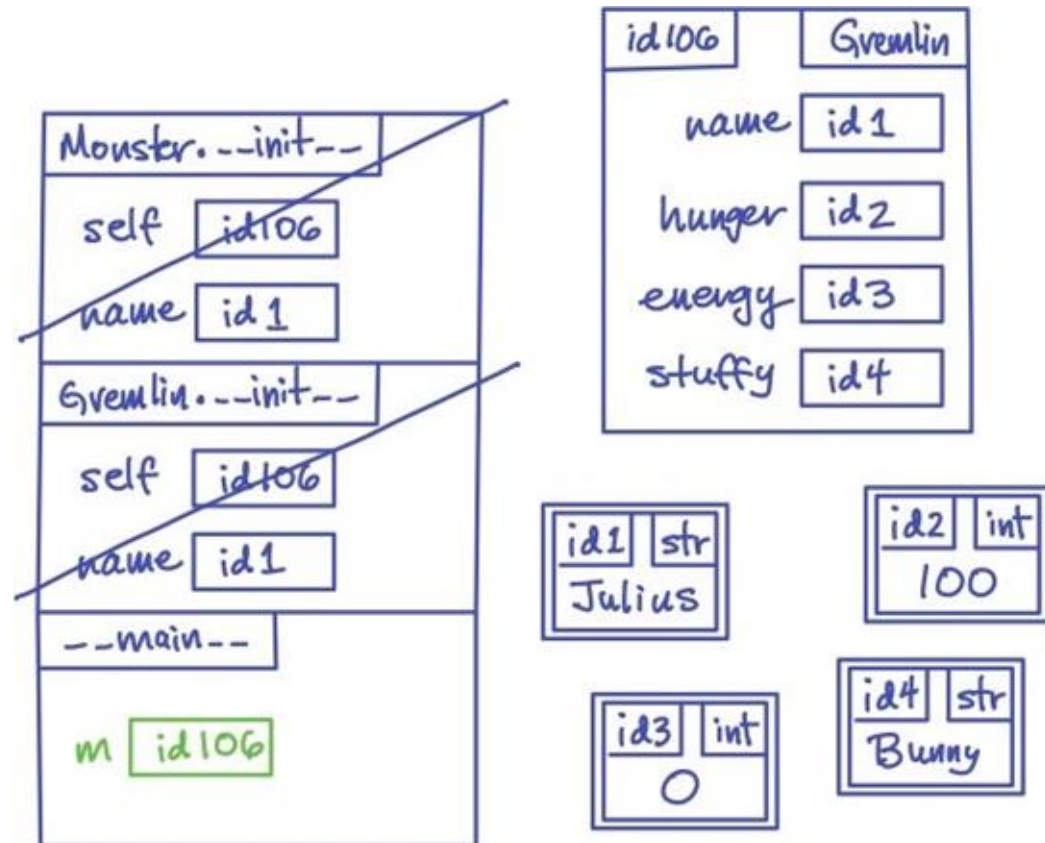


For another example: see [3.7](#)

```
class Monster:
    def __init__(self, name: str) -> None:
        self.name = name
        self.hunger = 100
        self.energy = 0

class Gremlin(Monster):
    def __init__(self, name: str) -> None:
        Monster.__init__(self, name)
        self.stuffy = 'Bunny'

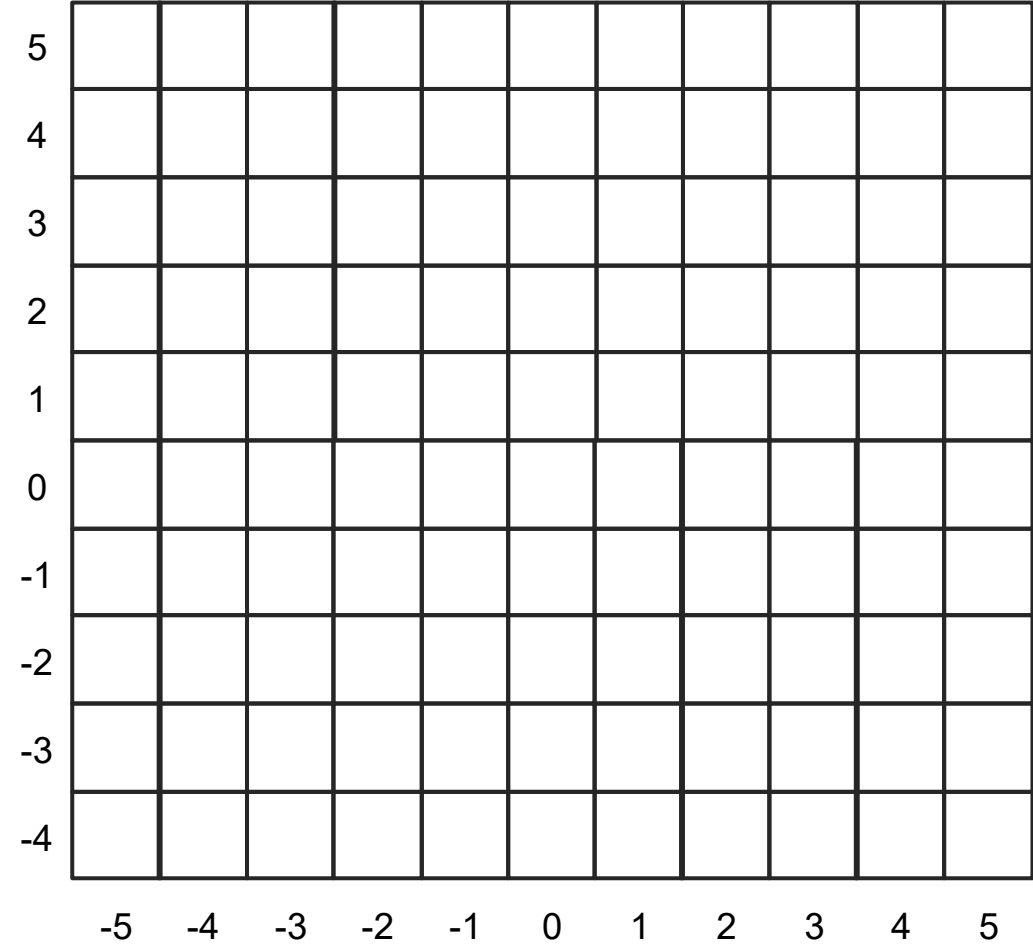
if __name__ == '__main__':
    m = Gremlin('Julius')
```



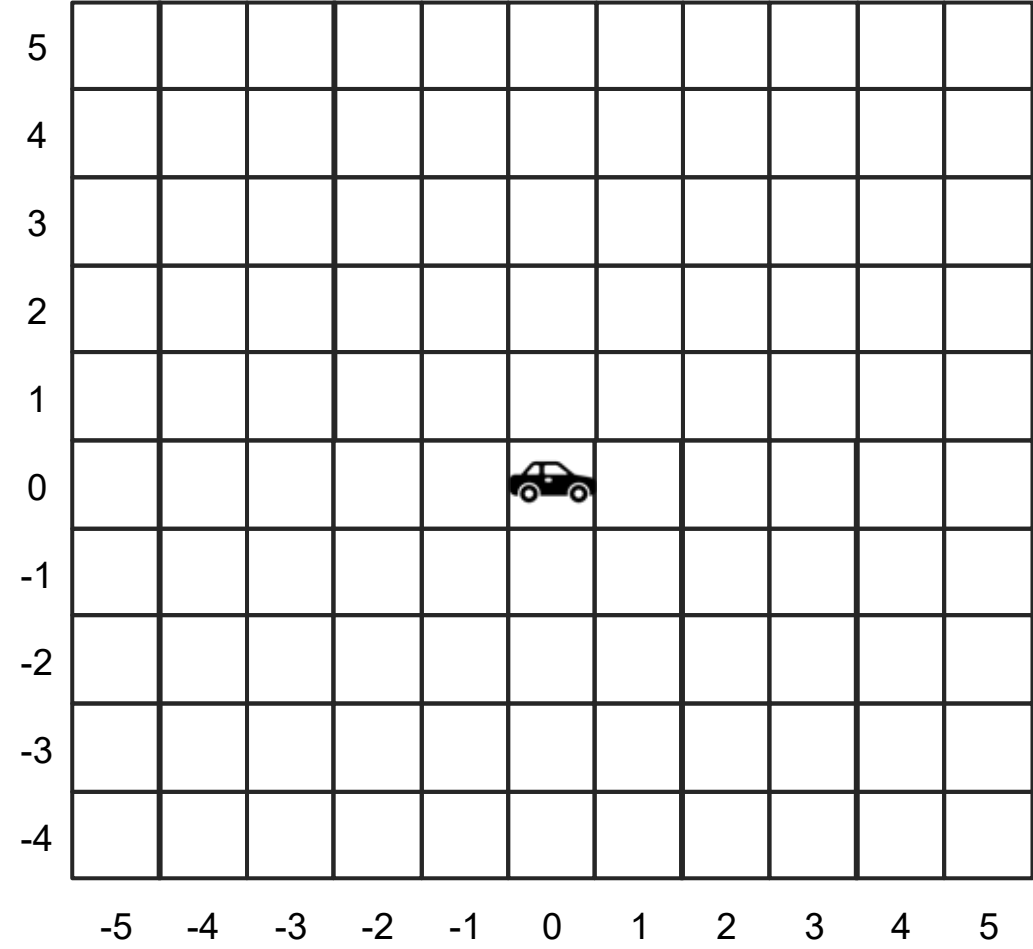
Worksheet: Inheritance

- ❑ Super class: Vehicle
 - ❑ Subclasses: Car, Helicopter, UnreliableMagicCarpet
- ❑ Another class: SuperDuperManager
 - ❑ Keeps track of Vehicles (of any type!)
 - ❑ Calls various Vehicle methods

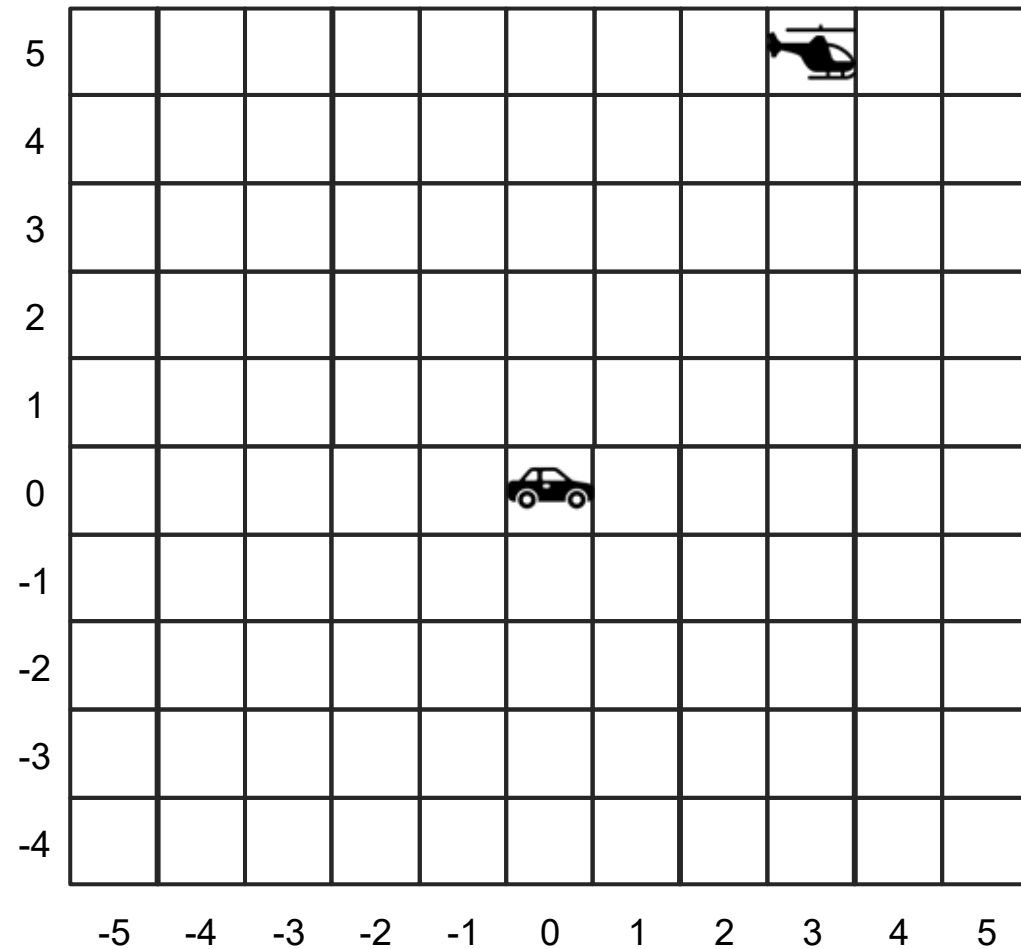
Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	✗	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



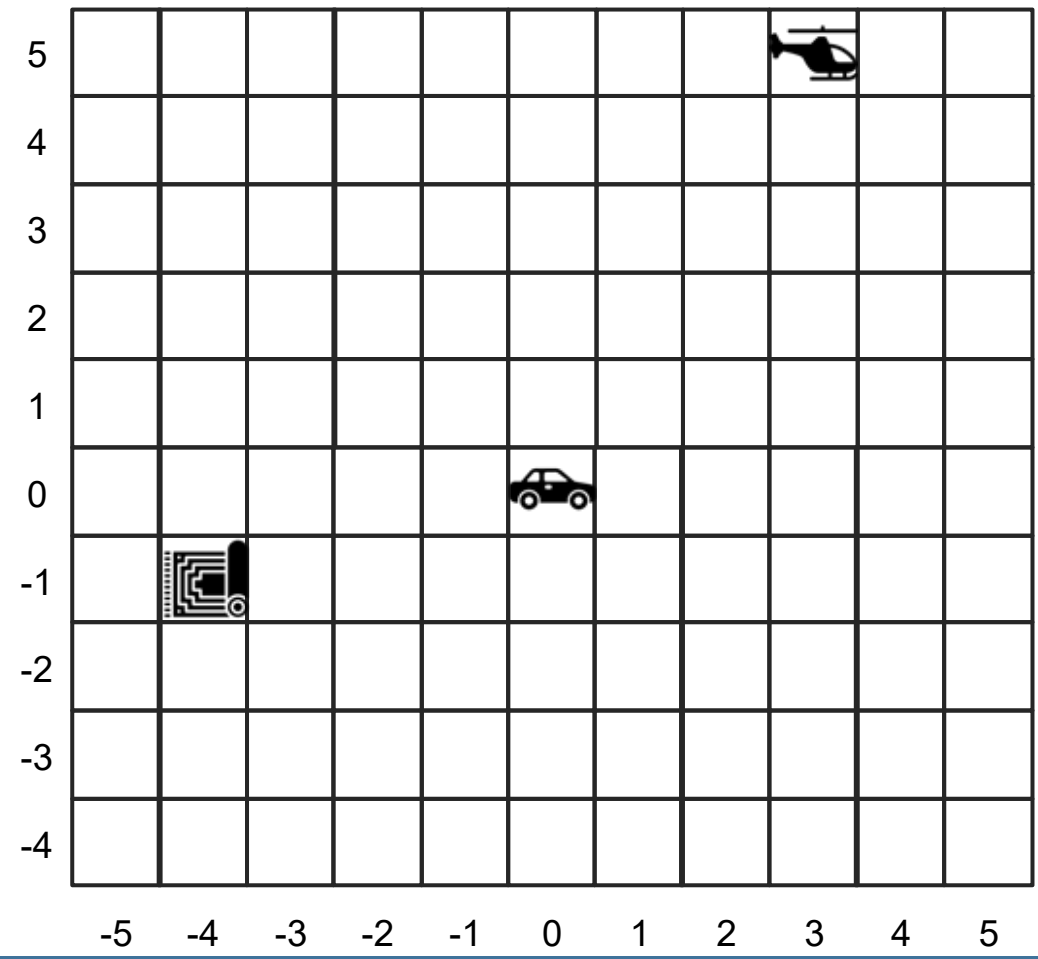
Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	✗	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



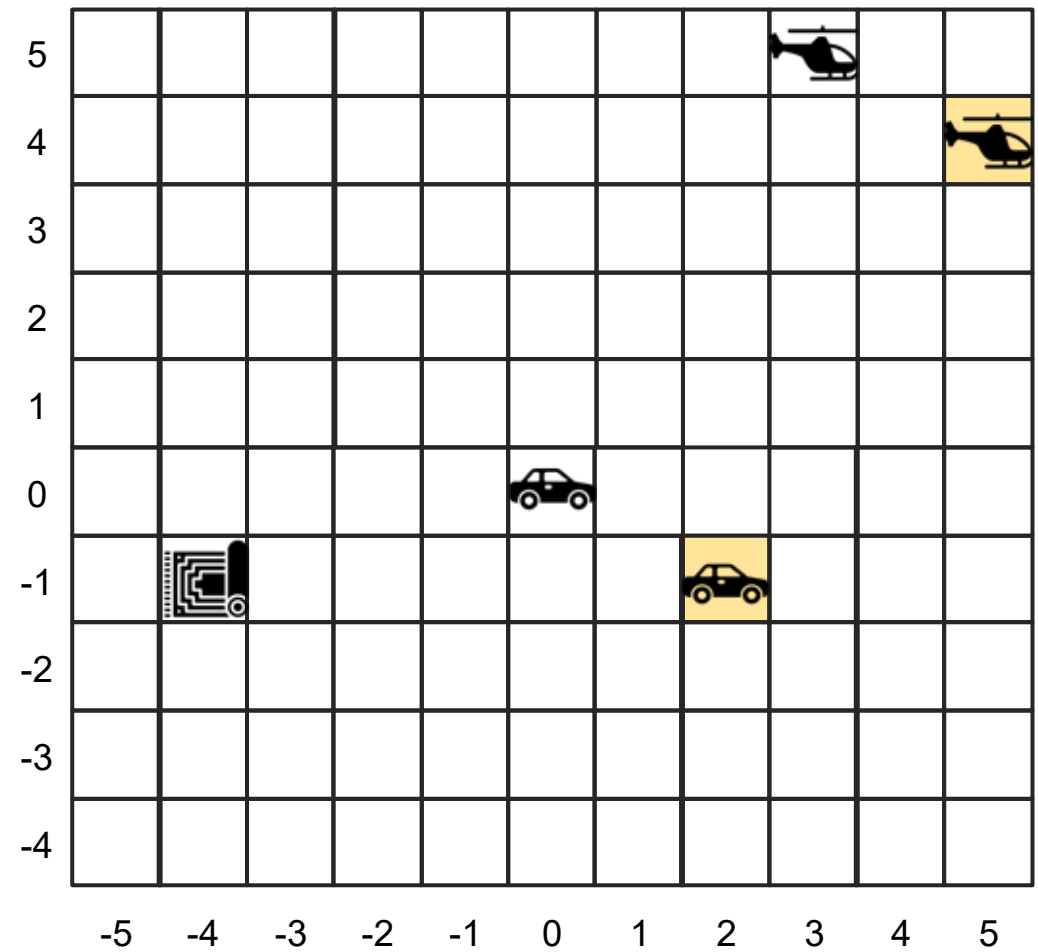
Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	✗	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



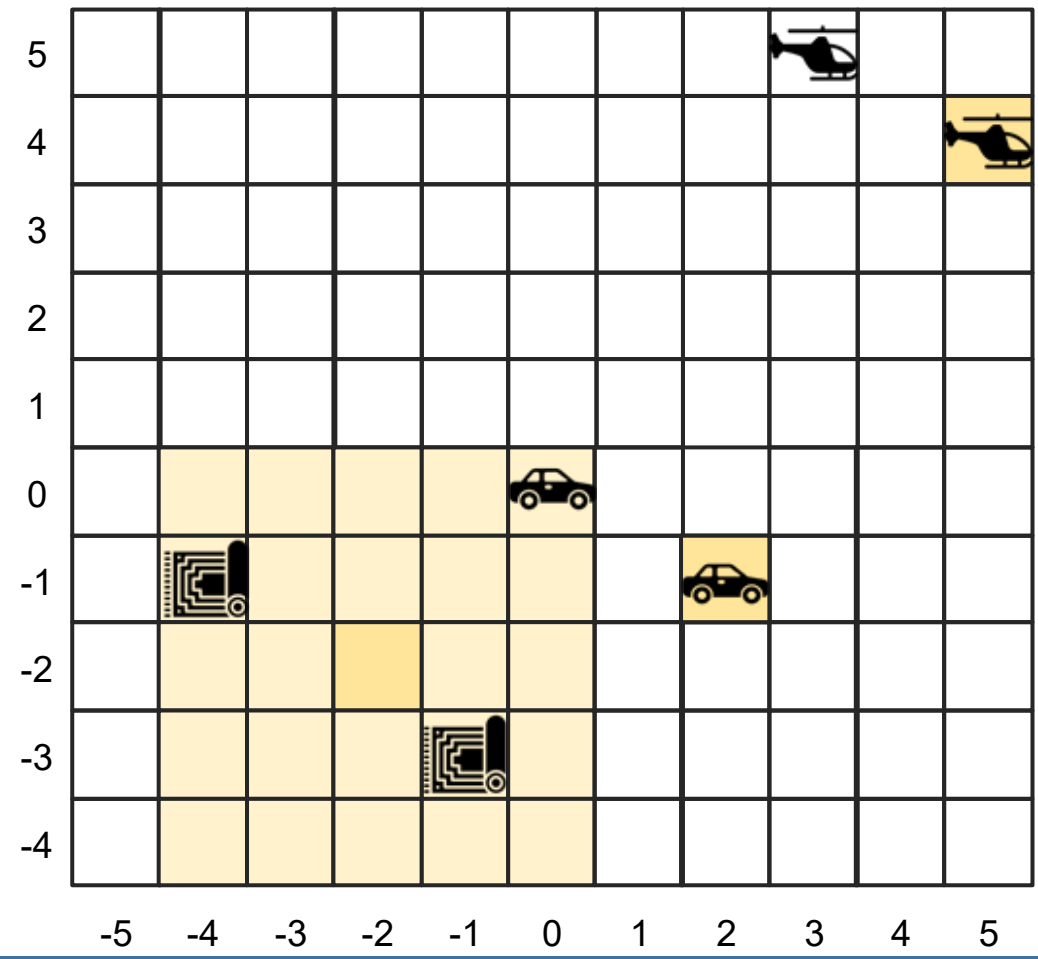
Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	✗	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



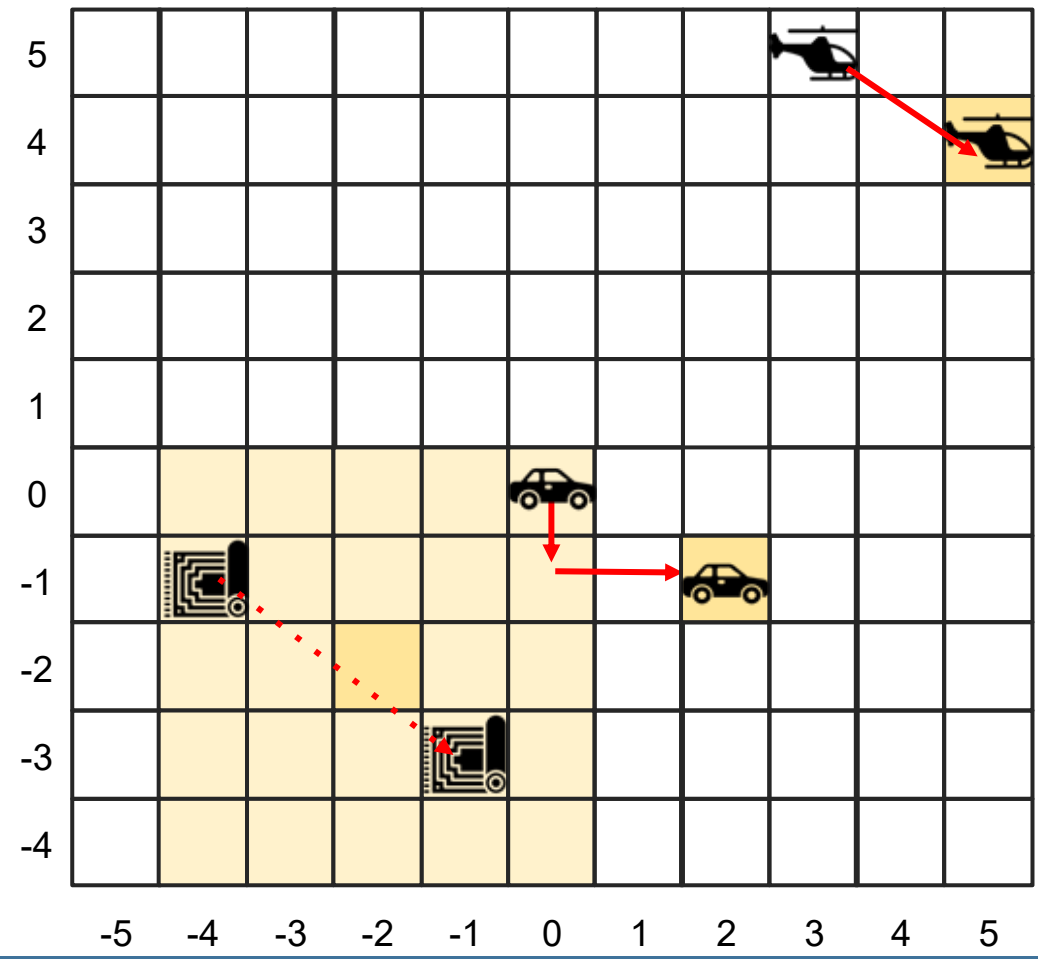
Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	✗	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



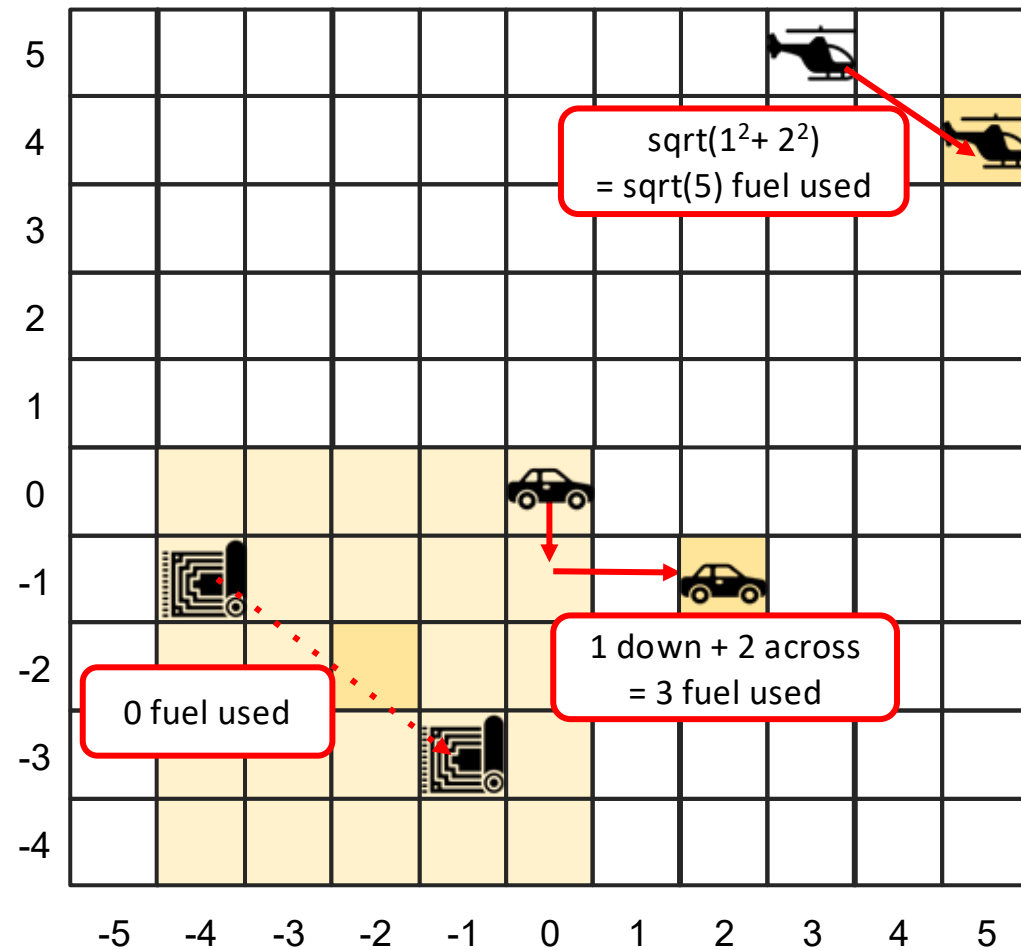
Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	✗	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	X	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



Type	Initial position	Moves to	Can move diagonally?	Fuel usage
car	(0, 0)	the desired location	✗	1 unit fuel per unit distance
helicopter	(3, 5), the launchpad	the desired location	✓	1 unit fuel per unit distance
unreliable magic carpet	a random location*	a random location*	✓	uses no fuel



1. What are the two classes defined in the starter code, and what additional classes will you need to create for this exercise? Identify any inheritance relationships between them.

2. Read the `Vehicle` class and answer these questions:

(a) What instance attributes does every vehicle possess?

(b) Does every vehicle start with the same amount of fuel and the same position?

(c) Why is method `fuel_needed` not implemented?

(d) After reading class `Vehicle`, what method(s) do you know for sure must be defined in each of its subclasses?

3. Class `SuperDuperManager` keeps track of all the vehicles. Read the header and docstring for each of its methods. Write doctest examples for the class that use each method at least once. (If the docstrings are well written, you should *not* have to read method bodies to be confident that you know what each method does!)

```
class Vehicle:
    """An abstract class for a vehicle in the Super Duper system.

    === Attributes ===
    position:
        The coordinates of this vehicle on a grid.
    fuel:
        The amount of fuel remaining for this vehicle.

    === Representation invariants ===
    - fuel >= 0
    """
    position: Tuple[int, int]
    fuel: int

    def __init__(self, initial_fuel: int,
                  initial_position: Tuple[int, int]) -> None:
        """Initialize a new Vehicle with the given fuel and position.

        Precondition: initial_fuel >= 0
        """
        self.fuel = initial_fuel
        self.position = initial_position

    def fuel_needed(self, new_x: int, new_y: int) -> int:
        """Return how much fuel would be needed to move to the given position.

        Note: the amount returned may be larger than self.fuel,
        indicating that this vehicle may not move to the given position.
        """
        raise NotImplementedError

    def move(self, new_x: int, new_y: int) -> None:
        """Move this vehicle to a new position.

        Do nothing if this vehicle does not have enough fuel to move to the specified position.
        """
        needed = self.fuel_needed(new_x, new_y)
        if needed <= self.fuel:
            self.position = (new_x, new_y)
            self.fuel -= needed
```


1. What are the two classes defined in the starter code, and what additional classes will you need to create for this exercise? Identify any inheritance relationships between them.

Vehicle and SuperDuperManager have been defined.
We need Car, Helicopter, and UnreliableMagicCarpet, all of which inherit from Vehicle.

2. Read the `Vehicle` class and answer these questions:

- (a) What instance attributes does every vehicle possess?
- (b) Does every vehicle start with the same amount of fuel and the same position?
- (c) Why is method `fuel_needed` not implemented?
- (d) After reading class `Vehicle`, what method(s) do you know for sure must be defined in each of its subclasses?

3. Class `SuperDuperManager` keeps track of all the vehicles. Read the header and docstring for each of its methods. Write doctest examples for the class that use each method at least once. (If the docstrings are well written, you should *not* have to read method bodies to be confident that you know what each method does!)

```
class Vehicle:
    """An abstract class for a vehicle in the Super Duper system.

    === Attributes ===
    position:
        The coordinates of this vehicle on a grid.
    fuel:
        The amount of fuel remaining for this vehicle.

    === Representation invariants ===
    - fuel >= 0
    """
    position: Tuple[int, int]
    fuel: int

    def __init__(self, initial_fuel: int,
                  initial_position: Tuple[int, int]) -> None:
        """Initialize a new Vehicle with the given fuel and position.

        Precondition: initial_fuel >= 0
        """
        self.fuel = initial_fuel
        self.position = initial_position

    def fuel_needed(self, new_x: int, new_y: int) -> int:
        """Return how much fuel would be needed to move to the given position.

        Note: the amount returned may be larger than self.fuel,
        indicating that this vehicle may not move to the given position.
        """
        raise NotImplementedError

    def move(self, new_x: int, new_y: int) -> None:
        """Move this vehicle to a new position.

        Do nothing if this vehicle does not have enough fuel to move to the specified position.
        """
        needed = self.fuel_needed(new_x, new_y)
        if needed <= self.fuel:
            self.position = (new_x, new_y)
            self.fuel -= needed
```

1. What are the two classes defined in the starter code, and what additional classes will you need to create for this exercise? Identify any inheritance relationships between them.

Vehicle and SuperDuperManager have been defined.
We need Car, Helicopter, and UnreliableMagicCarpet, all of which inherit from Vehicle.

2. Read the `Vehicle` class and answer these questions:

(a) What instance attributes does every vehicle possess?

fuel and position

(b) Does every vehicle start with the same amount of fuel and the same position?

(c) Why is method `fuel_needed` not implemented?

(d) After reading class `Vehicle`, what method(s) do you know for sure must be defined in each of its subclasses?

3. Class `SuperDuperManager` keeps track of all the vehicles. Read the header and docstring for each of its methods. Write doctest examples for the class that use each method at least once. (If the docstrings are well written, you should *not* have to read method bodies to be confident that you know what each method does!)

```
class Vehicle:
    """An abstract class for a vehicle in the Super Duper system.

    === Attributes ===
    position:
        The coordinates of this vehicle on a grid.
    fuel:
        The amount of fuel remaining for this vehicle.

    === Representation invariants ===
    - fuel >= 0
    """
    position: Tuple[int, int]
    fuel: int

    def __init__(self, initial_fuel: int,
                  initial_position: Tuple[int, int]) -> None:
        """Initialize a new Vehicle with the given fuel and position.

        Precondition: initial_fuel >= 0
        """
        self.fuel = initial_fuel
        self.position = initial_position

    def fuel_needed(self, new_x: int, new_y: int) -> int:
        """Return how much fuel would be needed to move to the given position.

        Note: the amount returned may be larger than self.fuel,
        indicating that this vehicle may not move to the given position.
        """
        raise NotImplementedError

    def move(self, new_x: int, new_y: int) -> None:
        """Move this vehicle to a new position.

        Do nothing if this vehicle does not have enough fuel to move to the specified position.
        """
        needed = self.fuel_needed(new_x, new_y)
        if needed <= self.fuel:
            self.position = (new_x, new_y)
            self.fuel -= needed
```

1. What are the two classes defined in the starter code, and what additional classes will you need to create for this exercise? Identify any inheritance relationships between them.

Vehicle and SuperDuperManager have been defined.
We need Car, Helicopter, and UnreliableMagicCarpet, all of which inherit from Vehicle.

2. Read the `Vehicle` class and answer these questions:

(a) What instance attributes does every vehicle possess?

fuel and position

(b) Does every vehicle start with the same amount of fuel and the same position?

No: Car starts at (0, 0), Helicopter starts at (3, 5), and UnreliableMagicCarpet starts anywhere between (-10, -10) and (10, 10). Their initial fuel is whatever's provided.

(c) Why is method `fuel_needed` not implemented?

(d) After reading class `Vehicle`, what method(s) do you know for sure must be defined in each of its subclasses?

3. Class `SuperDuperManager` keeps track of all the vehicles. Read the header and docstring for each of its methods. Write doctest examples for the class that use each method at least once. (If the docstrings are well written, you should *not* have to read method bodies to be confident that you know what each method does!)

```
class Vehicle:
    """An abstract class for a vehicle in the Super Duper system.

    === Attributes ===
    position:
        The coordinates of this vehicle on a grid.
    fuel:
        The amount of fuel remaining for this vehicle.

    === Representation invariants ===
    - fuel >= 0
    """
    position: Tuple[int, int]
    fuel: int

    def __init__(self, initial_fuel: int,
                  initial_position: Tuple[int, int]) -> None:
        """Initialize a new Vehicle with the given fuel and position.

        Precondition: initial_fuel >= 0
        """
        self.fuel = initial_fuel
        self.position = initial_position

    def fuel_needed(self, new_x: int, new_y: int) -> int:
        """Return how much fuel would be needed to move to the given position.

        Note: the amount returned may be larger than self.fuel,
        indicating that this vehicle may not move to the given position.
        """
        raise NotImplementedError

    def move(self, new_x: int, new_y: int) -> None:
        """Move this vehicle to a new position.

        Do nothing if this vehicle does not have enough fuel to move to the specified position.
        """
        needed = self.fuel_needed(new_x, new_y)
        if needed <= self.fuel:
            self.position = (new_x, new_y)
            self.fuel -= needed
```

1. What are the two classes defined in the starter code, and what additional classes will you need to create for this exercise? Identify any inheritance relationships between them.

Vehicle and SuperDuperManager have been defined.
We need Car, Helicopter, and UnreliableMagicCarpet, all of which inherit from Vehicle.

2. Read the `Vehicle` class and answer these questions:

(a) What instance attributes does every vehicle possess?

fuel and position

(b) Does every vehicle start with the same amount of fuel and the same position?

No: Car starts at (0, 0), Helicopter starts at (3, 5), and UnreliableMagicCarpet starts anywhere between (-10, -10) and (10, 10). Their initial fuel is whatever's provided.

(c) Why is method `fuel_needed` not implemented?

We don't know what a standard "Vehicle" should calculate fuel needed! All our Vehicles calculate fuel differently.

(d) After reading class `Vehicle`, what method(s) do you know for sure must be defined in each of its subclasses?

3. Class `SuperDuperManager` keeps track of all the vehicles. Read the header and docstring for each of its methods. Write doctest examples for the class that use each method at least once. (If the docstrings are well written, you should *not* have to read method bodies to be confident that you know what each method does!)

```
class Vehicle:
    """An abstract class for a vehicle in the Super Duper system.

    === Attributes ===
    position:
        The coordinates of this vehicle on a grid.
    fuel:
        The amount of fuel remaining for this vehicle.

    === Representation invariants ===
    - fuel >= 0
    """
    position: Tuple[int, int]
    fuel: int

    def __init__(self, initial_fuel: int,
                  initial_position: Tuple[int, int]) -> None:
        """Initialize a new Vehicle with the given fuel and position.

        Precondition: initial_fuel >= 0
        """
        self.fuel = initial_fuel
        self.position = initial_position

    def fuel_needed(self, new_x: int, new_y: int) -> int:
        """Return how much fuel would be needed to move to the given position.

        Note: the amount returned may be larger than self.fuel,
        indicating that this vehicle may not move to the given position.
        """
        raise NotImplementedError

    def move(self, new_x: int, new_y: int) -> None:
        """Move this vehicle to a new position.

        Do nothing if this vehicle does not have enough fuel to move to the specified position.
        """
        needed = self.fuel_needed(new_x, new_y)
        if needed <= self.fuel:
            self.position = (new_x, new_y)
            self.fuel -= needed
```


1. What are the two classes defined in the starter code, and what additional classes will you need to create for this exercise? Identify any inheritance relationships between them.

Vehicle and SuperDuperManager have been defined.
We need Car, Helicopter, and UnreliableMagicCarpet, all of which inherit from Vehicle.

2. Read the `Vehicle` class and answer these questions:

(a) What instance attributes does every vehicle possess?

fuel and position

(b) Does every vehicle start with the same amount of fuel and the same position?

No: Car starts at (0, 0), Helicopter starts at (3, 5), and UnreliableMagicCarpet starts anywhere between (-10, -10) and (10, 10). Their initial fuel is whatever's provided.

(c) Why is method `fuel_needed` not implemented?

We don't know what a standard "Vehicle" should calculate fuel needed! All our Vehicles calculate fuel differently.

(d) After reading class `Vehicle`, what method(s) do you know for sure must be defined in each of its subclasses?

`fuel_needed`

3. Class `SuperDuperManager` keeps track of all the vehicles. Read the header and docstring for each of its methods. Write doctest examples for the class that use each method at least once. (If the docstrings are well written, you should *not* have to read method bodies to be confident that you know what each method does!)

```
class Vehicle:
    """An abstract class for a vehicle in the Super Duper system.

    === Attributes ===
    position:
        The coordinates of this vehicle on a grid.
    fuel:
        The amount of fuel remaining for this vehicle.

    === Representation invariants ===
    - fuel >= 0
    """
    position: Tuple[int, int]
    fuel: int

    def __init__(self, initial_fuel: int,
                  initial_position: Tuple[int, int]) -> None:
        """Initialize a new Vehicle with the given fuel and position.

        Precondition: initial_fuel >= 0
        """
        self.fuel = initial_fuel
        self.position = initial_position

    def fuel_needed(self, new_x: int, new_y: int) -> int:
        """Return how much fuel would be needed to move to the given position.

        Note: the amount returned may be larger than self.fuel,
        indicating that this vehicle may not move to the given position.
        """
        raise NotImplementedError

    def move(self, new_x: int, new_y: int) -> None:
        """Move this vehicle to a new position.

        Do nothing if this vehicle does not have enough fuel to move to the specified position.
        """
        needed = self.fuel_needed(new_x, new_y)
        if needed <= self.fuel:
            self.position = (new_x, new_y)
            self.fuel -= needed
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

`class SuperDuperManager:`
 """A class responsible for keeping track of all vehicles in the system.
 Attributes:
 - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
 For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
 """
 _vehicles: dict[str, Vehicle]

 def __init__(self) -> None:
 """Initialize a new SuperDuperManager.

 There are no vehicles in the system when first created.
 """
 self._vehicles = {}

 def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
 """Add a new vehicle with the given type, id_, and fuel to the system.

 Do nothing if there is already a vehicle with the given id.

 Preconditions:
 - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
 - fuel >= 0
 """
 # Check to make sure the identifier isn't already used.
 if id_ not in self._vehicles:
 if vehicle_type == 'Car':
 self._vehicles[id_] = Car(fuel)
 elif vehicle_type == 'Helicopter':
 self._vehicles[id_] = Helicopter(fuel)
 elif vehicle_type == 'UnreliableMagicCarpet':
 self._vehicles[id_] = UnreliableMagicCarpet(fuel)

 def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
 """Move the vehicle with the given id.

 The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
 Do nothing if there is no vehicle with the given id,
 or if the corresponding vehicle does not have enough fuel to move.
 """
 if id_ in self._vehicles:
 self._vehicles[id_].move(new_x, new_y)

 def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
 """Return the position of the vehicle with the given id.

 Return a tuple of the (x, y) position of the vehicle.
 Return None if there is no vehicle with the given id.
 """
 if id_ in self._vehicles:
 return self._vehicles[id_].position
 return None

 def get_vehicle_fuel(self, id_: str) -> int | None:
 """Return the amount of fuel of the vehicle with the given id.

 Return None if there is no vehicle with the given id.
 """
 if id_ in self._vehicles:
 return self._vehicles[id_].fuel
 return None

~20 minutes

Only work on Q8 if you have time :)

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
        - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
            For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
    - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
        For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```


4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
    - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
        For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
        - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
            For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
        - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
            For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

**`__init__`, `fuel_needed`, and `move`
(`fuel_needed` is inherited, even if it's not implemented!)**

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
        - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
            For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```


4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

`__init__`, `fuel_needed`, and `move`
(`fuel_needed` is inherited, even if it's not implemented!)

(b) Which of these inherited methods must it implement because its parent did not?

`fuel_needed`

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
        - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
                      For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

`__init__`, `fuel_needed`, and `move`

(`fuel_needed` is inherited, even if it's not implemented!)

(b) Which of these inherited methods must it implement because its parent did not?

`fuel_needed`

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

`__init__`: It needs to set the initial position to (0, 0) by calling the parent.
(i.e. call it as a helper)

`fuel_needed`: Not implemented, need to override and implement it.

`move` is fine – we don't need to change anything in it!

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system.
    Attributes:
        - _vehicles: Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
            For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    """
    _vehicles: dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - vehicle_type in {'Car', 'Helicopter', 'UnreliableMagicCarpet'}
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> tuple[int, int] | None:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position
        return None

    def get_vehicle_fuel(self, id_: str) -> int | None:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
        return None
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # == Private Attributes ==
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```


4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # === Private Attributes ===
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```


4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # == Private Attributes ==
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # == Private Attributes ==
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # === Private Attributes ===
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```


4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # === Private Attributes ===
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

`__init__`, `fuel_needed`, and `move`
(`fuel_needed` is inherited, even if it's not implemented!)

(b) Which of these inherited methods must it implement because its parent did not?

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # === Private Attributes ===
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```

4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

`__init__`, `fuel_needed`, and `move`
(`fuel_needed` is inherited, even if it's not implemented!)

(b) Which of these inherited methods must it implement because its parent did not?

`fuel_needed`

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # === Private Attributes ===
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```


4. Find the code that keeps track of all the vehicles.

(a) What instance attribute is used to keep track of the vehicles? What is the type of this attribute?

(b) Where is it initialized?

(c) Where is it updated?

5. You are about to implement class `Car` (on a separate sheet of paper). Let's figure out the attributes.

(a) If you just accept the initializer that `Car` inherits from class `Vehicle`, what instance attributes will every car possess?

fuel and position

(b) Does class `Car` need any other instance attributes?

No! We just need our fuel and position.

6. Now let's figure out the methods of class `Car`.

(a) What method(s) does `Car` inherit?

`__init__`, `fuel_needed`, and `move`

(`fuel_needed` is inherited, even if it's not implemented!)

(b) Which of these inherited methods must it implement because its parent did not?

`fuel_needed`

(c) Which of these inherited methods must it override because the inherited implementation is not adequate? In these cases, should it call the parent class method as a helper, to get part of the work done?

`__init__`: It needs to set the initial position to (0, 0) by calling the parent.
(i.e. call it as a helper)

`fuel_needed`: Not implemented, need to override and implement it.

`move` is fine – we don't need to change anything in it!

7. With all of this sorted out, the code is much easier to write. Implement class `Car`.

8. Use the same strategy as in questions 5 and 6 to implement class `Helicopter` and class `UnreliableMagicCarpet`.

```
class SuperDuperManager:
    """A class responsible for keeping track of all vehicles in the system."""
    # == Private Attributes ==
    # _vehicles:
    #     Maps a string that uniquely identifies a vehicle to the corresponding Vehicle object.
    #     For example, _vehicles['car1'] would be a Vehicle object with the id_ 'car1'.
    _vehicles: Dict[str, Vehicle]

    def __init__(self) -> None:
        """Initialize a new SuperDuperManager.

        There are no vehicles in the system when first created.
        """
        self._vehicles = {}

    def add_vehicle(self, vehicle_type: str, id_: str, fuel: int) -> None:
        """Add a new vehicle with the given type, id_, and fuel to the system.

        Do nothing if there is already a vehicle with the given id.

        Preconditions:
        - <vehicle_type> is one of 'Car', 'Helicopter', or 'UnreliableMagicCarpet'.
        - fuel >= 0
        """
        # Check to make sure the identifier isn't already used.
        if id_ not in self._vehicles:
            if vehicle_type == 'Car':
                self._vehicles[id_] = Car(fuel)
            elif vehicle_type == 'Helicopter':
                self._vehicles[id_] = Helicopter(fuel)
            elif vehicle_type == 'UnreliableMagicCarpet':
                self._vehicles[id_] = UnreliableMagicCarpet(fuel)

    def move_vehicle(self, id_: str, new_x: int, new_y: int) -> None:
        """Move the vehicle with the given id.

        The vehicle called <id_> should be moved to position (<new_x>, <new_y>).
        Do nothing if there is no vehicle with the given id,
        or if the corresponding vehicle does not have enough fuel to move.
        """
        if id_ in self._vehicles:
            self._vehicles[id_].move(new_x, new_y)

    def get_vehicle_position(self, id_: str) -> Optional[Tuple[int, int]]:
        """Return the position of the vehicle with the given id.

        Return a tuple of the (x, y) position of the vehicle.
        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].position

    def get_vehicle_fuel(self, id_: str) -> Optional[int]:
        """Return the amount of fuel of the vehicle with the given id.

        Return None if there is no vehicle with the given id.
        """
        if id_ in self._vehicles:
            return self._vehicles[id_].fuel
```

Abstract classes — interfaces

An abstract class is first and foremost the explicit representation of an **interface** in a Python program.

□ What do we mean by that?

A watch



Interface

Implementation



Advantages of separating these:

- Wearer: don't need to understand the mechanism in order to use the watch.
- Maker: can change the mechanism and everyone still knows how to use the watch.

A function

Interface: defined by the function header and docstring

Implementation: the function body

Advantages of separating these:

- Client: don't need to understand the body in order to use the function.
- Implementer: can change the implementation and all client code still works.

A class

Interface: defined by the public attributes and methods

Implementation: the private attributes, private methods, and bodies of all methods.

Advantages of separating these: as before.

A class hierarchy

Interface: the shared public interface defined by the parent class

Advantages of separating these: as before, plus:

- Client: don't even need to know what kind you have!
- Implementer: can even define new kinds and all client code still works!
- This is monumentally powerful.

Clocks



- All able to "tell time"
 - Track "hours" and "minutes"
- Are all clocks
- Different implementations



Example from our payroll example

```
employees: list[Employee]  
for emp in self.employees:  
    emp.pay(date.today())
```



We don't know what type this is, but we do know:

- It is some kind of Employee.
- So it has a pay method, because every subclass inherits that. The Employee class defines a **common public interface**.

Example from SuperDuperManager

```
_vehicles: dict[str, Vehicle]  
self._vehicles[id].move(new_x, new_y)
```



We don't know what type this is, but we do know:

- It is some kind of Vehicle.
- So it has a move method, because every subclass inherits that. The Vehicle class defines a **common public interface**.

Polymorphism

```
_vehicles: dict[str, Vehicle]  
self._vehicles[id].move(new_x, new_y)
```

We say that the highlighted expression is **polymorphic**.

- poly: many; morph: form
- The expression can take many forms.
It can refer to a Car, an UnreliableMagicCarpet, even a subclass of vehicle that has not been defined yet!

Polymorphism

```
_vehicles: dict[str, Vehicle]  
self._vehicles[id].move(new_x, new_y)
```

Polymorphic!

vs.

```
if type(self._vehicles[id]) == Car:  
    self._vehicles[id].move(new_x, new_y)
```

Not polymorphic!

We know that this *has* to be of type Car, so it can only take one form!

```
elif ...
```

Class design decisions with inheritance

- ❑ What attributes and methods should comprise the **shared public interface**?
- ❑ For each method, should its implementation be **shared or separate** for each subclass?

“Is a” vs. “Has a”

- ❑ Don't forget about composition! Inheritance is only one kind of relationship between classes and is often *not* appropriate to describe the logical relationship between the entities you want to model.

“Is a” vs. “Has a”

- "A House *has* People"

- Implies composition: a House class that contains Person objects

- "A Person *is a* Mammal"

- Implies inheritance: a Person subclass inheriting from the Mammal superclass ("anything a Mammal can do, a human can do")

The four cases of method inheritance

1. Subclass inherits an implemented method.

The four cases of method inheritance

1. Subclass inherits an implemented method.

Example:

`Car` inherited `Vehicle.move()` without changing it at all!

The four cases of method inheritance

2. Subclass overrides an abstract method (to implement it).

The four cases of method inheritance

2. Subclass overrides an abstract method (to implement it).

Example:

`fuel_needed` was an abstract method, and all subclasses had to implement it.

The four cases of method inheritance

3. Subclass overrides an implemented method
(to *replace* it)

The four cases of method inheritance

3. Subclass overrides an implemented method
(to *replace* it)

Example:

`UnreliableMagicCarpet.move()` can be written in a way that replaces `Vehicle.move()`

The four cases of method inheritance

4. Subclass overrides an implemented method
(to *extend* it)

The four cases of method inheritance

4. Subclass overrides an implemented method
(to *extend* it)

Example:

`__init__()` methods tend to be written in this way (calling on the parent's `__init__`, and then adding to it.)

What is an "instance"?

An object that belongs to a class. Example:

```
c = Car(5)
```

- ❑ "c is a Car" also means "c is an instance of a Car"
- ❑ c is of type Car

isinstance vs. type

`c = Car(5)`

- ❑ The type of `c` is `Car`
- ❑ `c` is a `Car` → `c` is an instance of `Car`
- ❑ Cars are Vehicles
 - ❑ `c` is a `Vehicle` → `c` is an instance of `Vehicle`
- ❑ `c` is *not* of type `Vehicle`!
 - ❑ Type is more specific
- ❑ `isinstance` includes type and all parents/ancestors

object

```
c = Car(5)
```

- ❑ Furthermore: `c` is an object → `c` is an instance of `object`
- ❑ Every class is an object in Python
- ❑ Defines default behaviour for classes
 - ❑ We inherit some methods from `object`
 - ❑ These let us write things like `str(obj)`, `a == b`, etc.